ISTA 130

Classes II

What do hiring managers value most?



Class Book

The class definition begins with a header:

```
class <classname>:
```

- (this defines a new *type*)

- A **class** definition is a compound statement (like **def**, **while**, **if**, ...)
  - The name of your class should begin with a capital letter:

```
class Book:
```

- We create an *instance* by *calling* the class:

```
my_book = Book()
```

  - We'll refer to this as calling the *constructor* function: it constructs new instances

`class Book` – What Does `init` Return?

For our Book class **__init__** will look like this:

```
def __init__(self, the_title):
    self.title = the_title
    return None
```

- We see the dot-operator applied to `self`

  self.title = the_title

- Clearly we're assigning the value of `the_title` to something, but what?
- If the statement looked like this:

  ```
  x = the_title
  ```

  we'd know it meant to create a new variable called `x` and assign the value of the `the_title` to `x`

Class Book

```
def __init__(self, the_title):
    self.title = the_title
    return None
```

- The **self** parameter will automatically refer to the new unborn Book *instance*
- **self.title** indicates a variable named `title` found inside the Book instance being constructed:
  - we're giving *each* new Book its *own* variable called `title`
  - and assigning it the value of the parameter `the_title`
  - think of the dot-operator as a possessive form, e.g: `self.title` means `self`'s `title` variable
  - we'll refer to such a variable as an *attribute* or an *instance variable*

Instance Variables

- Each object has its own instance variables.
- The values of the instance variables are what makes two objects of the same type different. That, and their location in memory.

Making Objects/Instances

- Call the name of the class as though it is the name of a function:
  - `my_book = Book("The Hobbit")`
- Python then does a lot behind the scenes – calls `new` and `init`.

Improving The `Book` Class

To make the Book more useful we'll add more *attributes* (also known as *instance variables*):

- add an attribute for the **author**
- add an attribute for the **price**
- add an attribute for the **quantity** in stock
- The `__init__` method's parameters are different variables from the instance variables
  - It's common practice to give them the same name

    ```
    self.title = title
    ```

  - Don't let this confuse you

    `title` is the **parameter**

    `self.title` is the **instance's own variable** by the same name!

Magic Methods

- Python has a number of built-in methods it recognizes
  - Their names begin and end with 2 underscores
    - e.g. `__init__`
  - We won't call these methods directly.
  - We just define them for Python to use.
- We defined an `__init__` method in our class like this:

```
def __init__(self):
```

  - we never call the `__init__` method in our code
  - Python uses the `__init__` method to initialize any instance we create
  - first parameter to `__init__` will always be `self`
  - `self` parameter refers to the particular instance being created

Improving The `Book` Class

- We'll now define a `__repr__` method in our class like this:

```
def __repr__(self):
```

  - we won't call the `__repr__` method directly in our code
  - `Python` uses the `__repr__` method when it needs a string representation of an instance
  - The `__repr__` method will always have `self` as its only parameter
  - The `self` parameter refers to the particular instance the `__repr__` method is being called with.

The `__repr__` Method

Python expects the `__repr__` method to take one parameter (`self`) and to *return* a string:

- For example, if we want this:

```
print(my_book)
```

- To print a book's title followed by the author's name in parentheses we could define it like this:

```
def __repr__(self):
    return self.title + " (" + self.author + ")"
```

- Now any time we pass a Book instance to `print`, Python will automatically call our `__repr__` method and give the returned value to `print`

Add Another Method

- What if we'd like to know the total value of our inventory for a particular book
  - our Book instances each have a price and a quantity in stock
  - so we could just do this in main:

```
my_book.price * my_book.quantity
```

  - but, if we'll be doing this often we should go ahead and write a method:

```
def total_value(self)
    return self.price * self.quantity
```

Add Another Method

- Whenever we call a method, for example:

```
hobbit.total_value()
```

- Imagine the book instance saying to itself:

"Hey, I think I'll call my `total_value` method!"

```
def total_value(self)
    return self.price * self.quantity
```

- The `self` parameter refers to the particular instance that called the method on itself.
  - This time self will refer to the zorba instance:

```
zorba.total_value()
```

Add Another Method

```
def total_value(self)
    return self.price * self.quantity
```

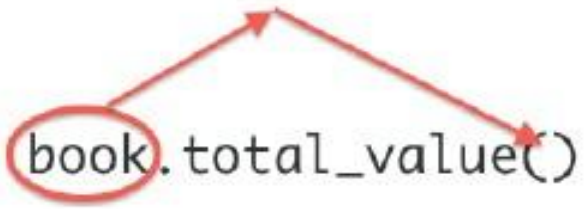
- When we call the method like this:

```
book.total_value()
```

the `self` parameter will refer to whatever Book instance the `book` variable refers to

- It might help to imagine it like this:

at the moment of the method call
the sneaky little book instance leaps over the dot operator
landing inside the parentheses becoming the first argument



Add Another Method

- Now imagine we'd like to print out a detailed report about our inventory including for each book:
  - number in stock, price, title, author, total stock value
  - we could iterate over all books collecting this information into strings to print
  - or we could just write a new method and have each instance call the method
- Let's add a `print_report` method that will print the required information for a Book
  - we'll use string formatting to make it pretty
- Notice the call to `self.total_value()` from within the `print_report` method
  - here, just like anywhere else, we can use an instance to call a method

Add a Function Outside the Class Definition

- We can still write functions like we normally do:
  - We just need to make sure we write them *outside* of the class definition
  - How does `Python` know when some code is contained by other code?
    - indentation
- Imagine we want a function that will print a report on each book in our inventory followed by the total value of all books in our inventory.

Book Example Review

Last time we looked at a class used to represent books for a bookstore program:

- The class `Book` had the following methods:
  - `__init__` `# initialization method`
  - `__repr__` `# string representation`
  - `total_value` `# return price * quantity`
  - `print_report` `# print information about a book`
- We also had a function outside of the class:
  - `print_book_inventory`

Book Example Continued

As we buy and sell books we'll need to change the state of our book objects.

- For example, we'll need to increment/decrement the `quantity` attribute
- First we'll try just accessing the attributes directly.
- If you need more control over how an attribute can be changed, write a method.
- Methods that change the value in an attribute are often called "mutators".

Book Example Continued

Not all attributes are initialized by parameters to the **__init__** method.

- Let's add an attribute to keep track of the minimum number of copies of each book we'd like to have in stock
  - This value will likely be different for each book (depending on how fast they each sell)
  - We probably won't know the best number for a given book until we've seen how well the book sells
  - We'll set the number to 5 for all books initially
  - Later on we'll want to change it for each book
- What code will we need to change/add?

Book Example Continued

- Assume we want to begin selling music albums in our store
- How should we represent an album in our program?
- What information should we store about each album?
  - band name
  - album name
  - price
  - song list?
  - quantity in stock?
  - ...

Many Books

- Let's put all of our books in a list.